INITIATION PYTHON · TP EN GROUPE DE 4 · LUNDI 9 NOVEMBRE

# TP · Diagnostic de dossier AstraCorp

L'équipe informatique d'AstraCorp veut un outil simple pour surveiller ses dossiers : combien de fichiers, de quels types, et surtout lesquels prennent trop de place. Votre groupe écrit ce script, à quatre, dans un seul fichier : `diagnostic.py`.

## L'objectif

Le script doit :

1. demander un chemin de dossier à l'utilisateur ;
2. parcourir **tous** les fichiers, y compris ceux des sous-dossiers ;
3. compter les `.txt`, les `.log` et les autres ;
4. détecter les fichiers de plus de 5 Mo ;
5. afficher un rapport dans le terminal ;
6. enregistrer le résultat dans un fichier CSV, qui garde l'historique de tous les diagnostics.

Voici ce qu'on doit obtenir à la fin, sur un dossier qui contient un gros fichier :

Le rapport final (exemple)

```
$ python3 diagnostic.py
Chemin du dossier à diag : bac-a-sable

=== Diagnostic dossier ===
Dossier diag : /home/alice/python-semaine/bac-a-sable
Nombre de fichiers total : 16
Nombre de fichiers .txt : 3
Nombre de fichiers .log : 1
Autres fichiers : 12

=== Fichiers de plus de 5 Mo ===
- video.mp4 - 6.0 Mo
Nombre de fichiers volumineux : 1

=== Diagnostic Final ===
Attention : Fichiers volumineux détectés !
```

## Qui fait quoi

| Élève | Parties | Mission |
|---|---|---|
| Élève 1 | 1 et 2 | les imports, la saisie du dossier, la vérification qu'il existe |
| Élève 2 | 3 et 4 | la liste des fichiers, le comptage par extension |
| Élève 3 | 5 et 6 | les fichiers volumineux, l'affichage du rapport |
| Élève 4 | 7 et 8 | le dossier de logs, l'export CSV |

C'est une suggestion : organisez-vous comme vous voulez. Mais **tout le monde lit la documentation et aide les autres**. Un groupe qui finit, c'est un groupe où chacun comprend tout le fichier, pas seulement sa partie.

Dans le code, les blocs marqués **fourni** sont à recopier tels quels : il suffit de les comprendre. Les blocs marqués **à vous** sont à écrire.

## La documentation

Gardez ces pages ouvertes dans le navigateur. Chaque lien est cliquable.

| Module | Ce qui vous servira |
|---|---|
| `pathlib`, les chemins, fichiers et dossiers | `Path.exists()`, `Path.is_file()`, `Path.rglob()`, `Path.suffix`, `Path.stat()`, `Path.mkdir()`, l'opérateur `/` |
| `datetime`, la date et l'heure | `datetime.now()`, `strftime()` |
| `csv`, les fichiers tableur | `csv.writer` |
| les fichiers | `open()` et `with` |
| les listes en une ligne | les compréhensions de liste |

Une partie de la documentation française n'est pas encore traduite et reste en anglais. Les exemples de code, eux, se comprennent dans toutes les langues.

## Partie 1 · Imports et constantes ÉLÈVE 1

Un `import` permet d'utiliser du code déjà prêt dans Python. On a besoin de trois modules de la bibliothèque standard.

```python
# FOURNI
from pathlib import Path
from datetime import datetime
import csv

# 5 Mo = 5 x 1024 x 1024 octets (1 Ko = 1024 octets, 1 Mo = 1024 Ko)
SIZE_LIMIT = 5 * 1024 * 1024
```

`SIZE_LIMIT` est en majuscules : c'est la convention pour une constante, une valeur qu'on ne modifie jamais.

## Partie 2 · Demander le dossier et vérifier qu'il existe ÉLÈVE 1

`Path(...)` transforme une chaîne de caractères en objet « chemin », qui sait faire beaucoup de choses : dire s'il existe, lister son contenu, donner sa taille.

- **La doc** : `Path`, `input`, `Path.exists()`

```python
# FOURNI
directory = Path(input("Chemin du dossier à diag : "))
```

**À vous :**

1. vérifier que le dossier existe, avec `directory.exists()` ;
2. s'il n'existe pas : afficher « Dossier introuvable », puis arrêter le programme avec `exit()`.

```python
# Indice
if not directory.exists():
    ...
```

## Partie 3 · Lister tous les fichiers ÉLÈVE 2

`rglob("*")` parcourt le dossier **et** tous ses sous-dossiers. Le `*` veut dire « tout ce qui existe ». On ne garde que les **fichiers**, pas les dossiers, grâce à `is_file()`.

- **La doc** : `Path.rglob()`, `Path.is_file()`, les compréhensions de liste

```
# FOURNI : à ne pas réécrire, juste à comprendre et utiliser
files = [f for f in directory.rglob("*") if f.is_file()]
```

En français : « pour chaque élément `f` trouvé par `rglob`, si c'est un fichier, on le met dans la liste `files` ». C'est exactement la boucle avec `append` que vous savez écrire, en une ligne :

```
files = []
for f in directory.rglob("*"):
    if f.is_file():
        files.append(f)
```

## Partie 4 · Compter les fichiers par extension ÉLÈVE 2

On prépare des compteurs, qui commencent à 0, et une liste vide pour les gros fichiers.

```
# FOURNI
total = 0
txt_count = 0
log_count = 0
others_count = 0
big_files = []   # liste de tuples (chemin, taille), utilisée en partie 5
```

- **La doc** : `Path.suffix`

**À vous**, pour **chaque** fichier `f` de la liste `files` :

1. ajouter 1 à `total` ;
2. récupérer l'extension : `ext = f.suffix.lower()`. `.suffix` donne `".txt"`, `".log"`, ou `""` s'il n'y a pas d'extension, et `.lower()` passe en minuscules, pour que `RAPPORT.TXT` compte aussi ;
3. si `ext` vaut `".txt"`, ajouter 1 à `txt_count` ; sinon, si elle vaut `".log"`, ajouter 1 à `log_count` ; sinon, ajouter 1 à `others_count`.

Ne vous occupez pas encore de la taille : c'est la partie de l'élève 3.

```
# Indice de structure
for f in files:
    total += 1
    ext = f.suffix.lower()
    if ext == ".txt":
        ...
    elif ext == ".log":
        ...
    else:
        ...
```

## Partie 5 · Détecter les fichiers volumineux ÉLÈVE 3

La taille d'un fichier, en octets, s'obtient avec `f.stat().st_size`.

- **La doc** : `Path.stat()`, `st_size`

**À vous**, pour chaque fichier `f` :

1. `size = f.stat().st_size` ;
2. si `size` dépasse `SIZE_LIMIT`, ajouter le tuple `(f, size)` à la liste : `big_files.append((f, size))`. Attention aux **doubles** parenthèses : on ajoute **un** tuple de deux valeurs.

> **Astuce de groupe.** Vous pouvez fusionner les parties 4 et 5 dans une seule boucle `for`, pour ne parcourir `files` qu'une fois. Mettez-vous d'accord avec l'élève 2.

## Partie 6 · Afficher le rapport ÉLÈVE 3

- **La doc** : les f-strings, `Path.resolve()` (le chemin complet), `round()`, `len()`, `.name`

```
# FOURNI, à compléter
print("\n=== Diagnostic dossier ===")
print(f"Dossier diag : {directory.resolve()}")
print(f"Nombre de fichiers total : {total}")
# print(f"Nombre de fichiers .txt : ...")
# print(f"Nombre de fichiers .log : ...")
# print(f"Autres fichiers : ...")
```

**À vous :**

1. compléter les trois `print` commentés ;
2. afficher le titre `=== Fichiers de plus de 5 Mo ===`, puis :
   - si `big_files` est vide : « Aucun fichier volumineux détecté » ;
   - sinon, pour chaque `(f, size)` de `big_files`, calculer `size_mo = round(size / (1024 * 1024), 2)` et afficher `- {f.name} - {size_mo} Mo`, puis le nombre de fichiers volumineux, `len(big_files)` ;
3. afficher le titre `=== Diagnostic Final ===`, puis « Dossier propre ! » s'il n'y a aucun gros fichier, ou « Attention : Fichiers volumineux détectés ! » sinon.

Le `\n` au début des titres saute une ligne avant : le rapport respire.

## Partie 7 · Préparer le dossier de logs ÉLÈVE 4

On crée le dossier `AstraCorp/logs` s'il n'existe pas encore. `parents=True` crée aussi les dossiers parents manquants (ici `AstraCorp`), et `exist_ok=True` évite de planter si le dossier existe déjà.

- **La doc** : `Path.mkdir()`, l'opérateur `/`

```
# FOURNI
logs_dir = Path("AstraCorp/logs")
logs_dir.mkdir(parents=True, exist_ok=True)

csv_path = logs_dir / "diagnostic_global.csv"
```

L'opérateur `/` de `pathlib` assemble des chemins proprement : c'est mieux que de coller des `/` à la main, et ça marche aussi sous Windows, où le séparateur est `\`.

## Partie 8 · Écrire une ligne dans le CSV ÉLÈVE 4

Le fichier s'ouvre en mode `"a"`, pour *append* : on **ajoute** à la fin, sans écraser l'historique. `newline=""` est recommandé avec le module `csv`, et `encoding="utf-8"` gère les accents.

- **La doc** : `open()`, `csv.writer`, `writerow()`, `strftime()`

```
# FOURNI : l'ouverture du fichier et le writer
file_exist = csv_path.exists()   # True si le CSV existe déjà

with open(csv_path, "a", newline="", encoding="utf-8") as f:
    writer = csv.writer(f)
    # ... votre code ici, décalé dans le bloc with
```

**À vous**, à l'intérieur du bloc `with` :

1. si le fichier n'existait **pas** encore (`not file_exist`), écrire la ligne d'en-tête, avec le nom des colonnes : `["Date", "Dossier", "Total Fichiers", "Nb .txt", "Nb .log", "Autres fichiers", "Fichiers volumineux"]`
2. puis, **toujours**, écrire une ligne de données avec la date et l'heure actuelles formatées, `directory.resolve()`, `total`, `txt_count`, `log_count`, `others_count`, et `len(big_files)`.

```
# Indice
if not file_exist:
    writer.writerow([...])
writer.writerow([...])
```

Pour la date du moment : `datetime.now().strftime("%Y-%m-%d %H:%M:%S")` donne par exemple `2026-11-09 14:35:01`.

| Code | Signification | Code | Signification |
|---|---|---|---|
| `%Y` | l'année, sur 4 chiffres | `%H` | l'heure, de 00 à 23 |
| `%m` | le mois | `%M` | les minutes |
| `%d` | le jour | `%S` | les secondes |

## Tester

**1. Un petit dossier de test.** Le `bac-a-sable` des exercices de jeudi est parfait. S'il n'existe plus : `python3 creer_bac_a_sable.py`. Ajoutez-lui un gros fichier et un fichier `.log`, pour tester tous les cas :

Deux fichiers de plus dans le bac à sable

```
$ truncate -s 6M bac-a-sable/photos/video.mp4
$ touch bac-a-sable/erreurs.log
```

Le diagnostic du bac à sable

```
$ python3 diagnostic.py
Chemin du dossier à diag : bac-a-sable

=== Diagnostic dossier ===
Dossier diag : /home/alice/python-semaine/bac-a-sable
Nombre de fichiers total : 16
Nombre de fichiers .txt : 3
Nombre de fichiers .log : 1
Autres fichiers : 12

=== Fichiers de plus de 5 Mo ===
- video.mp4 - 6.0 Mo
Nombre de fichiers volumineux : 1

=== Diagnostic Final ===
Attention : Fichiers volumineux détectés !
```

**2. Vérifiez les compteurs** à l'écran. Pour comparer, les mêmes comptes en bash :

Les mêmes comptes, en bash

```
$ find bac-a-sable -type f | wc -l
16
$ find bac-a-sable -type f -name "*.txt" | wc -l
3
$ find bac-a-sable -type f -size +5M
bac-a-sable/photos/video.mp4
```

**3. Relancez une deuxième fois.** Le CSV doit avoir **deux** lignes de données, et l'en-tête **une seule** fois.

L'historique après deux lancements

```
$ cat AstraCorp/logs/diagnostic_global.csv
Date,Dossier,Total Fichiers,Nb .txt,Nb .log,Autres fichiers,Fichiers volumineux
2026-11-08 20:25:08,/home/alice/python-semaine/bac-a-sable,16,3,1,12,1
2026-11-08 20:25:10,/home/alice/python-semaine/bac-a-sable,16,3,1,12,1
```

**4. Un dossier qui n'existe pas** doit afficher « Dossier introuvable », sans traceback.

Un chemin inexistant

```
$ python3 diagnostic.py
Chemin du dossier à diag : nulle-part
Dossier introuvable
```

**5. Ouvrez le CSV** dans LibreOffice Calc : chaque diagnostic est une ligne du tableau.

Pour fabriquer vous-mêmes des cas de test, deux commandes bash utiles : `truncate -s 6M fichier.mp4` crée un fichier de 6 Mo, `touch erreurs.log` un fichier vide.

## Bonus, si vous avez fini

- Afficher aussi la **taille totale** de tous les fichiers, en Mo.
- Afficher le **chemin complet** des gros fichiers, pas seulement leur nom.
- Ajouter une colonne « Taille totale » dans le CSV. Attention : l'en-tête des anciens diagnostics n'aura pas cette colonne. Que faire du fichier existant ?
- Si l'utilisateur donne le chemin d'un **fichier** au lieu d'un dossier, le script affiche un rapport vide. Vérifiez que c'est bien un dossier avec `Path.is_dir()`.

- Relancez le diagnostic sur le dossier courant, `.` : le CSV d'AstraCorp apparaît dans les « autres fichiers ». Normal ? Comment l'exclure ?